S'orienter

Personnes Programmes

	Documents	Données
Formats	docx, txt	xlsx, csv, R-DB

Et json, HTML, XML, jpg, pdf ?

JSON
JavaScript Object Notation

Utilisé pour stocker et échanger des informations entre applications. Issu de JavaScript (orienté description d'objets)

Principes de la syntaxe

- Texte entre **{** et **}**
- paire **nom : valeur** (key : Value)
  "color": "Yellow"
- Les paires sont séparées par **,**
  "color": "Yellow", "age": 6

JSON
JavaScript Object Notation

Utilisé pour stocker et échanger des informations entre applications.

Principes de la syntaxe

- Texte entre { et }
- paire **nom : valeur** (key : Value)
  "color": "Yellow" /* toujours des " */
  "age" : 6 /* les chiffres sont sans " */
- Les paires sont séparées par **,**
  "color": "Yellow", "age": 6

Exemple d'un document JSON

{ "species": "Dog",
"race": "Poodle",
"color": "Yellow",
"age": 6
}

Note: c'est lisible par un humain et par un programme

Typages et imbrication

Types possibles:

- Objet entre { et } (avec possibilité d'imbrication)
- Nombres
- Chaines

```
// un objet JSON
{
"species": "Dog",
"breed": "Poodle",
"age": 6,                    Objet imbriqué
"traits": { "eyeColor": "brown",
        "coatColor": "yellow",
        "weight": "137lbs" }
}
```


Typages et imbrication

Types possibles:

- Objet (avec imbrication)
- Nombres
- Chaines
- Tableau [ ]
- true / false
- null

Arrays
{ "employees":["John", "Anna", "Peter"] }
Booleans
{"sale":true}
null
{"middlename":null}

JSON Schema

- Créer des "modèles" d'objets JSon

JSON Schema

- Définir la structure de document JSON
- Le vocabulaire de base
  - **Title** : Nom du schema JSon
  - **Description** : texte descriptif
  - **Type** : un nom de type, tel que "Object" (= un objet Json)
  - **Properties** : clés (noms), leur type associé, et autres.
  - **Items**: éléments d'un tableau (avec un type)
  - { } pour un objet
  - [ ] pour un tableau

JSON Schema

Les types des propriétés :

- string
- number
- boolean
- null
- object
- array

Exemple de JSON Schema

```
{
"$schema": "http://json-schema.org/draft-04/schema#",
"title": "Product",
"description": "A product from Acme's catalog",
"type": "object",
"properties": {"id":
                    { "description": "The unique identifier for a product",
                    "type": "integer" },
                "name":
                    { "description": "Name of the product",
                    "type": "string" },
                "price":
                    { "type": "number",
                    }
                },
"required": ["id", "price"]
}
```


Le document JSON

Code JSon (valide pour le schéma précédent)

```
[
  {
    "id": 2,
    "name": "An ice sculpture",
    "price": 12.50,
  },

  {
     "id": 3,
     "price": 25.50,
  }
]
```


Autres élements Json Schema

- **Minimum** : valeur minimum.
- **exclusiveMinimum** : si vrai, minimum exclusif ('>' et non '>=')
- **Maximum** : valeurs maximum
- **exclusiveMaximum** : si vrai, maximum exclusive ('<' et non '<=')

Exemple:

```
"price": {"type": "number",
          "minimum": 0,
          "exclusiveMinimum": true   // ne peut pas être 0
          }
```

Autres élements Json Schema

- **pattern** : une chaine qui respecte une expression régulière.

Exemple: les numéros de telephone d'Amérique du nord, tels que 555-1212“ ou "(888)555-1212“

```
{   "type": "string",
   "pattern": "^(\\([0-9]{3}\\))?[0-9]{3}-[0-9]{4}$“  }
```

Indications: ^ = au début ; $ = à la fin ;
?=bloc précédent optionnel ;
\\ = caractère suivant non interprété

Référence ici

Tableaux en Json Schema

{ "type": "array" }
pour [1, 2, 3, 4, 5] ou [3, "different", { "types" : "of values" }]
→ Éléments de différents types, nombre d'éléments non défini

Deux usages des tableaux:
Liste de validation: chaque item a un même type (utilise *items*)
Tuple : liste fixée d'items de types différents (utilise *prefixitems*)

Déclaration d'une liste de validation

```
{   "type": "array",
    "items": { "type": "number" }
}
```

[1, 2, 3]
~~[1, "2", 3]~~

Référence

Déclaration d'un tuple

```
{ "type": "array",
  "prefixItems": [
            { "type": "number" },
            { "type": "string" },
            { "enum": ["Street", "Avenue"] },
            { "enum": ["NW", "NE", "SW", "SE"] }
            ]
}
```

[1600, "Pennsylvania", "Avenue", "NW"]

Tableaux en Json Schema

Autres attributs des tableaux

- Sens de **Items** pour une déclaration de tuple:
  - Par défaut on peut ajouter des valeurs [1600, "Pennsylvania", "Avenue", "NW", "USA"]
    Mais si ajout de *"items": false*, alors il n'est pas autorisé d'ajouter des valeurs
  - Si ajout de *"items": { "type": "string" }* → seuls des champs de type string peuvent être ajoutés
- "**contains**": { "type": "number" } : le tableau doit contenir au moins un item de type nombre. (aussi : **minContains** et **maxContains**)
- Longueur d'un tableau: **minItems**, **maxItems**
- **uniqueItems** : Si vrai, alors unicité de chaque item du tableau

Référence à un schéma JSon

$id : donne un identifiant à un schéma
$ref: dans un schéma Json, fait référence à un autre schéma

```
{ "$id": "/schemas/address",
"type": "object",
"properties":
    {"street_address": { "type": "string" },
     "city": { "type": "string" },
     "state": { "type": "string" }
    },
 "required": ["street_address", "city", "state"] }
```

Fait référence à

```
{  "type": "object",
  "properties":
      {"last_name": { "type": "string" },
       "shipping_address": { "$ref": "/schemas/address" }
      },
  "required": ["last_name", "shipping_address"]
}
```


Bilan JSon

JSON, la syntaxe
JSON Schema, expression de modèles (vocabulaire et grammaire)

Important
Système ouvert = création, réutilisation, diffusion